equity_pricing

November 1, 2026

1 Equity pricing in OSEM

This notebook walks through how OSEM treats an equity position: from raw input data, to projected cash flows, to how the position moves through the main loop of a run. An equity share is defined by its issue date, dividend yield and frequency, market price and an assumed growth rate.

On a very high level:

1) Each equity position is saved into an EquityShare object.
2) A portfolio of EquityShare objects is then saved into the EquitySharePortfolio.
3) Before the main loop starts, the dividends and terminal value of each share are projected from its growth rate and stored in matrices, with the cash flow dates in the columns and individual shares in rows.
4) In every period of the main loop, the cash flows that have come due are credited to the bank account, the price of each share grows at its fixed growth rate, and the bank account is traded away proportionally.

A run does not price equities by discounting. Despite the title, `main.py` never computes a discounted cash flow price for a share: the price starts at the `Market_Price` from the input file and moves with the growth rate alone. The discounted cash flow price and the growth calibration exist in `EquityClasses.py` and are shown at the end of this notebook (Step 7 and Step 8), but nothing in a run calls them.

1.1 Table of contents

1. Methodology
   - Notation
   - Step 1 — Cash flow dates
   - Step 2 — Date fractions
   - Step 3 — Cash flow amounts
   - Step 4 — Crediting cash flows to the bank account
   - Step 5 — Growing the price
   - Step 6 — Liquidation and trading
   - Step 7 — Discounted cash flow price (not used in a run)
   - Step 8 — Calibrating the growth rate (not used in a run)
   - How the steps map onto the full OSEM run
2. Setup
   - Environment and imports
   - Run parameters
   - The risk-free curve

3. The equity portfolio
    - Loading the shares
    - Which fields drive a run
4. Steps 1 to 3 in numbers: cash flow dates and amounts
    - Dividend and terminal flows
    - Cash flow matrices
5. Steps 4 to 6 in numbers: the main loop
    - Running the equity leg
    - The price path
    - The units
6. Step 7 in numbers: discounted cash flow price
    - Price against the market price
    - Additivity of the spreads
7. Step 8 in numbers: calibrating the growth rate
8. Appendix: input data
    - Parameters.csv · EIOPA RFR files
    - Equity_Portfolio.csv

1.2 1. Methodology

What OSEM does with an equity position, step by step. No code in this section; the sections that follow run each step on the input portfolio. Steps 1 to 6 are what a run executes. Steps 7 and 8 are machinery that exists but that a run does not call.

1.2.1 Notation

This table defines the mathematical notation that will be used through the rest of the workbook

Symbol	Meaning
MD	modelling date
E	end of the modelling window (`settings.end_date`)
I	issue date
MP	market price at the modelling date
g	annual growth rate
dy	dividend yield, paid on **each** dividend date
f	dividend frequency (payments/year)
t_i^d, t_i	date of the i-th dividend, and its year fraction from MD
T	year fraction from MD to E
$MV(t)$	market value of one share t years after MD
div_i, $term$	i-th dividend and terminal value, per share
D_k	k-th projection date, $D_0 = MD$ and $D_n = E$
$P_{a,k}$	price of share a at D_k
$u_{a,k}$	units of share a at D_k
$y(t)$	risk-free spot yield for maturity t, from the calibrated EIOPA curve

Symbol	Meaning
s	combined discounting spread, used in Step 7 only

A note on f. The frequency sets only the *spacing* of the dividend dates, through `relativedelta(months=12 // f)`. It does **not** scale the dividend amount: dy is the yield paid on each dividend date, not an annualised yield, so a share with $dy = 0.01$ and quarterly frequency pays 1% of its market value four times a year. A share quoted with an annual yield y_{ann} paid f times a year is entered in `Equity_Portfolio.csv` with `Dividend_Yield` $= y_{ann}/f$.

1.2.2 Step 1 — Cash flow dates

Every share produces two kinds of cash flow: dividends and a terminal value.

A share has no maturity, so both run up to the end of the modelling window. The terminal value is paid once, on

$$t_T^d = E$$

Dividend dates:

$$MD \le t_1^d < t_2^d < \cdots < t_k^d \le E$$

The dividend grid is anchored on the issue date and stepped by $12/f$ months. Dates before MD are dropped, while a dividend falling exactly on MD is kept.

In numbers: section 4.

1.2.3 Step 2 — Date fractions relative to the modelling date

Each cash flow date is converted to a year fraction from the modelling date:

$$t_i = \frac{t_i^d - MD}{365.25}, \qquad T = \frac{E - MD}{365.25}$$

These fractions are always measured from MD, because the amounts in Step 3 are computed once, before the main loop starts.

In numbers: section 4.

1.2.4 Step 3 — Cash flow amounts

The market value of one share is assumed to grow at the constant rate g:

$$MV(t) = MP \cdot (1+g)^t$$

A dividend is a share of the market value on its payment date, and the terminal value is the market value at the end of the modelling window:

$$div_i = MV(t_i) \cdot dy \qquad\qquad term = MV(T)$$

The terminal value is the grown market value itself: no yield is applied to it, and it is not discounted. `create_terminal_flows` also takes a `terminal_rate`, which `main.py` sets to the ultimate forward rate, but it has no effect: `terminal_amount` returns the grown market value, with the Gordon growth formula `market_price / (terminal_rate - growth_rate)` left commented out.

All amounts are **per share**. They are computed once from the input growth rate and do not change during the run; the units held are applied only when a cash flow comes due in Step 4.

In numbers: section 4.

1.2.5 Step 4 — Crediting cash flows to the bank account

At each projection date D_k, every cash flow that has come due and has not been processed yet is credited to the bank account, using the units carried forward from the end of the previous period:

$$C_k^{div} = \sum_a u_{a,k} \sum_{i:\, D_{k-1} < t^d_{a,i} \le D_k} div_{a,i}$$

(in the first period, a dividend falling exactly on $D_0 = MD$ is included as well). The projection dates are anniversaries of the modelling date, so a dividend dated between two of them is credited on the next one, without any interest for the time in between.

`process_expired_cf` does this and drops the columns it has processed, so no cash flow is counted twice. The terminal values go through the same function and reach the bank account on the last date, $D_n = E$. `find_terminated_positions` runs just before it, to note which shares were paid out; Step 6 closes them.

In numbers: section 5.

1.2.6 Step 5 — Growing the price

After the cash flows, the price of each share is rolled forward with its growth rate:

$$P_{a,k} = P_{a,k-1} \cdot (1 + g_a)^{(D_k - D_{k-1})/365.25}, \qquad P_{a,0} = MP_a$$

The growth rate is fixed for the whole run: it is always read from the modelling date column of `eq_growth_df`. The period fractions add up, so $P_{a,k} = MP_a(1 + g_a)^{(D_k - MD)/365.25}$. The price in the loop therefore agrees with $MV(t)$ from Step 3, and the price on the last date equals the terminal value.

Paying a dividend does not lower the price. There is no ex-dividend drop, so a share earns its growth rate plus its dividends: roughly $g + f \cdot dy$ a year.

The period return is then the after-growth market value plus the dividends credited in Step 4, relative to the market value at the end of the previous period. Terminal values are left out of it (`portfolio_total_return`), because those shares are still in the market value at this point and adding them would count them twice.

In numbers: section 5.

1.2.7 Step 6 — Liquidation and trading

Two things change the units, and neither changes the price.

Liquidation. On the last date the units of the shares whose terminal value was paid are set to zero (`liquidate_positions`), so the value is held only as cash and not also as market value.

Trading. `trade` then drives the bank account towards zero by buying or selling the whole portfolio proportionally. With MV the market value of all equities and bonds:

- Cash surplus: $u_{a,k} \leftarrow u_{a,k}(1+p)$ with $p = \text{cash}/MV$
- Cash deficit: $u_{a,k} \leftarrow u_{a,k}(1-p)$ with $p = \min(1, -\text{cash}/MV)$

Every equity and bond is scaled by the same p, and the next period's cash flows are paid on these new units. If nothing is left to trade ($MV \leq 0$), as on the last date of a portfolio with no other assets, the cash stays in the bank account.

In numbers: section 5.

1.2.8 Step 7 — Discounted cash flow price (not used in a run)

`price_share` values the cash flows of Steps 1 to 3 off the risk-free curve, adding a parallel spread for the riskiness of the issuer:

$$PV = \sum_{i=1}^{k} \frac{div_i}{(1 + y(t_i) + s)^{t_i}} + \frac{term}{(1 + y(T) + s)^{T}}$$

The combined spread is the sum of three inputs, which all work in the same way:

$$s = s_{country} + s_{sector} + s_{stress}$$

- `spread_country` ($s_{country}$): extra riskiness related to the country of operation
- `spread_sector` (s_{sector}): extra riskiness of the sector in which the issuer operates
- `spread_stress` (s_{stress}): extra spread of an assumed shock scenario

These three fields are inputs to `price_share` only. A run never reads them, so changing them in `Equity_Portfolio.csv` has no effect on a run's results.

In numbers: section 6.

1.2.9 Step 8 — Calibrating the growth rate (not used in a run)

In a real world run, the growth rate is a modelling assumption brought in from outside. In a risk-neutral run, each asset should earn the risk-free rate. `bisection_growth` is the machinery for that: it searches for the growth rate at which the price of Step 7 equals the market price,

$$MP = PV(g)$$

over a bracket $[g_{\text{start}}, g_{\text{end}}]$, stopping when the bracket is narrower than `precision`. This is a tolerance on g, not on the price, so a small residual remains in the price.

Unlike the bond calibration, `bisection_growth` does not check that the bracket contains a solution. If it does not, the search walks to the upper end of the bracket and returns that bound as though it were a calibrated rate. If `max_iter` runs out, it returns the string `"Did not converge"` rather than raising an error.

This is not wired into a run: `run_type` is read from `Parameters.csv` into `Settings` and never consulted again, and growth rates are taken from the input file as given, so nothing currently switches a run to risk-neutral.

In numbers: section 7.

1.2.10 How the steps map onto the full OSEM run

Step	Code	When `main.py` runs it
1–3	`create_dividend_flows`, `create_terminal_flows`, `create_cashflow_dataframe`	Once, before the main loop
4	`process_expired_cf`, `find_terminated_positions`	Every period, first
5	`eq_price_df[current_date] = ...`, `portfolio_total_return`	Every period, after the cash flows
6	`liquidate_positions`, `trade`	Every period, last
7–8	`price_share`, `bisection_growth`	Never

The full run does more in each period: bond coupons, notionals and repricing, liability cash flows, and in `unit_linked` mode the policy mechanics between Steps 5 and 6. All of these move the bank account, which is why the equity units in a full run also change to fund liabilities and not only to reinvest dividends.

1.3 2. Setup

Prepares the environment, loads the run parameters and calibrates the risk-free curve. Nothing in this section is specific to equities.

1.3.1 Environment and imports

Locate the repository root (the folder holding `ALM.ini`), make the `osem` package importable from `src/`, and import the parts of OSEM this notebook uses.

[1]:
```
import datetime
import os
import sys
from dataclasses import replace
from pathlib import Path

import pandas as pd
```

```
# Walk up to the repository root, the folder holding ALM.ini
base_folder = next(path for path in [Path.cwd(), *Path.cwd().parents] if (path /
 ↪ "ALM.ini").is_file())
sys.path.insert(0, str(base_folder / "src"))
os.chdir(base_folder)
base_folder = os.getcwd()  # Get current working directory

from osem.ConfigurationClass import Configuration
from osem.CurvesClass import Curves
from osem.EquityClasses import EquityShare, EquitySharePortfolio
from osem.ImportData import get_configuration, get_equity_share, get_settings,␣
 ↪import_SWEiopa
from osem.MainLoop import (create_cashflow_dataframe,␣
 ↪find_terminated_positions, liquidate_positions,
                           portfolio_market_value, portfolio_total_return,␣
 ↪process_expired_cf,
                           set_dates_of_interest, trade)
```

1.3.2 Run parameters

`ALM.ini` gives the location of every input file:

[2]:
```
conf: Configuration
conf = get_configuration(os.path.join(base_folder, "ALM.ini"), os)
```

[3]:
```
parameters_file = conf.input_parameters
equity_portfolio_file = conf.input_equity_portfolio
```

[4]:
```
paramfile = pd.read_csv(parameters_file).set_index("Parameter")
```

The parameter file is:

---

[5]:
```
display(paramfile)
```

Parameter	Value
EIOPA_param_file	Input/Param_no_VA.csv
EIOPA_curves_file	Input/Curves_no_VA.csv
country	Slovenia
run_type	Risk Neutral
n_proj_years	50
Precision	1E-10
Tau	0.0001
compounding	-1
Modelling_Date	29/04/2023

```
liability_mode                  cashflow
random_seed                           42
```

The settings object holds data about file locations, information about the run settings and model parameters such as modelling date.

[6]:
```
settings = get_settings(parameters_file)
```

The run that the rest of the notebook works with:

---

[7]:
```
run_settings = pd.DataFrame(
    # Rendered as strings: in a mixed-type column pandas applies one float␣
 ↪format to every
    # numeric value, which would show a precision of 1e-10 as 0.0
    {"Value": [str(settings.modelling_date),
               str(settings.end_date),
               str(settings.n_proj_years),
               settings.country,
               f"{settings.precision:g}",
               f"{settings.tau:g}"]},
    index=["Modelling date",
           "End of modelling window",
           "Projection years",
           "Country",
           "Precision (curve calibration)",
           "Tau"],
)

display(run_settings)
```

```
                                Value
Modelling date                2023-04-29
End of modelling window       2073-04-29
Projection years                      50
Country                          Slovenia
Precision (curve calibration)       1e-10
Tau                                0.0001
```

1.3.3 The risk-free curve

`import_SWEiopa()` reads the necessary data about the current yield curve, and the `Curves` object holds the resulting term structure.

The equity leg of a run needs very little of it. `main.py` hands `curves.ufr` to `create_terminal_flows`, where it has no effect (see Step 3), and the projected curves are used

for bonds. In this notebook the curve is only needed for the discounted cash flow price and the calibration in sections 6 and 7.

[8]:
```
[maturities_country, curve_country, extra_param, Qb] = import_SWEiopa(settings.
 ↪EIOPA_param_file,
                                                                       settings.
 ↪EIOPA_curves_file, settings.country)
# Curves object with information about term structure
curves = Curves(extra_param["UFR"] / 100, settings.precision, settings.tau,␣
 ↪settings.modelling_date,
                settings.country)
```

[9]:
```
curves.set_observed_term_structure(maturity_vec=curve_country.index.
 ↪to_numpy(dtype=float), yield_vec=curve_country.to_numpy(dtype=float))
curves.calc_fwd_rates()
# The +1 matches main.py: the projected curve has to reach one year past the␣
 ↪end of the
# modelling window so the last projection period can still be priced.
curves.project_forward_rate(settings.n_proj_years+1)
curves.calibrate_projected(settings.n_proj_years+1, 0.05, 0.5, 1000)
```

1.4 3. The equity portfolio

1.4.1 Loading the shares

A Python generator reads the equity portfolio file and encodes it into a dictionary based on the asset id. Each asset id contains an EquityShare object describing a single equity position. The variable names from here on are the ones `main.py` uses.

[10]:
```
equity_input_generator = get_equity_share(equity_portfolio_file)
eq_input = {equity_share.asset_id: equity_share for equity_share in␣
 ↪equity_input_generator}
```

As an example, a single share in an EquityShare looks like this:

---

[11]:
```
display(eq_input[1125])
```

```
EquityShare(asset_id=1125, nace='A1.4.5', issuer=None, issue_date=datetime.
 ↪date(2021, 12, 3), dividend_yield=0.03, frequency=1, units=100.0,␣
 ↪market_price=123.01, growth_rate=0.01, spread_country=0.0, spread_sector=0.0,␣
 ↪spread_stress=0.0)
```

`EquitySharePortfolio` wraps a dictionary of these objects, keyed by asset id, and provides the portfolio-level methods used in the rest of the notebook.

[12]:
```
eq_ptf = EquitySharePortfolio(eq_input)
```

The whole equity portfolio at a glance. Note that `Annual yield` is derived, not an input: it is `Dividend yield` times `Frequency`, because the dividend yield is paid on each dividend date. Positions 6 to 29 repeat three share definitions, so their rows here and in the tables below repeat as well.

---

[13]:
```
portfolio_overview = pd.DataFrame(
    [{"Issue date": share.issue_date,
      "Dividend yield": share.dividend_yield,
      "Frequency": int(share.frequency),
      "Annual yield": share.dividend_yield * int(share.frequency),
      "Growth rate": share.growth_rate,
      "Units": share.units,
      "Market price": share.market_price}
     for share in eq_ptf.equity_share.values()],
    index=list(eq_ptf.equity_share),
)

display(portfolio_overview)
```

	Issue date	Dividend yield	Frequency	Annual yield	Growth rate	Units	\
1125	2021-12-03	0.03	1	0.03	0.01	100.0	
2123	2021-12-03	0.05	1	0.05	0.02	100.0	
3232	2019-12-03	0.04	1	0.04	0.04	100.0	
1	2021-12-03	0.03	1	0.03	0.01	100.0	
2	2021-12-03	0.05	1	0.05	0.02	100.0	
3	2019-12-03	0.04	1	0.04	0.04	100.0	
4	2021-12-03	0.03	1	0.03	0.01	100.0	
5	2021-12-03	0.05	1	0.05	0.02	100.0	
6	2019-12-03	0.04	1	0.04	0.04	100.0	
7	2021-12-03	0.03	1	0.03	0.01	100.0	
8	2021-12-03	0.05	1	0.05	0.02	100.0	
9	2019-12-03	0.04	1	0.04	0.04	100.0	
10	2021-12-03	0.03	1	0.03	0.01	100.0	
11	2021-12-03	0.05	1	0.05	0.02	100.0	
12	2019-12-03	0.04	1	0.04	0.04	100.0	
13	2021-12-03	0.03	1	0.03	0.01	100.0	
14	2021-12-03	0.05	1	0.05	0.02	100.0	
15	2019-12-03	0.04	1	0.04	0.04	100.0	
16	2021-12-03	0.03	1	0.03	0.01	100.0	
17	2021-12-03	0.05	1	0.05	0.02	100.0	
18	2019-12-03	0.04	1	0.04	0.04	100.0	
19	2021-12-03	0.03	1	0.03	0.01	100.0	
20	2021-12-03	0.05	1	0.05	0.02	100.0	
21	2019-12-03	0.04	1	0.04	0.04	100.0	
22	2021-12-03	0.03	1	0.03	0.01	100.0	
23	2021-12-03	0.05	1	0.05	0.02	100.0	
24	2019-12-03	0.04	1	0.04	0.04	100.0	

25	2021-12-03	0.03	1	0.03	0.01	100.0
26	2021-12-03	0.05	1	0.05	0.02	100.0
27	2019-12-03	0.04	1	0.04	0.04	100.0
28	2021-12-03	0.03	1	0.03	0.01	100.0
29	2021-12-03	0.05	1	0.05	0.02	100.0
30	2019-12-03	0.04	1	0.04	0.04	100.0

	Market price
1125	123.01
2123	12.12
3232	56.54
1	77.00
2	78.00
3	1.23
4	3.12
5	45.12
6	96.00
7	94.00
8	92.00
9	96.00
10	94.00
11	92.00
12	96.00
13	94.00
14	92.00
15	96.00
16	94.00
17	92.00
18	96.00
19	94.00
20	92.00
21	96.00
22	94.00
23	92.00
24	96.00
25	94.00
26	92.00
27	96.00
28	94.00
29	92.00
30	96.65

1.4.2 Which fields drive a run

Only six of the `EquityShare` fields affect a run. The rest are loaded from the portfolio file but play no part in it.

`EquityShare` field	CSV column	Used in a run	Role
`asset_id`	`Asset_ID`	–	Key of the portfolio dictionary and row index of every DataFrame
`issue_date`	`Issue_Date`	Yes	Anchors the dividend grid (I, Step 1)
`frequency`	`Frequency`	Yes	Spacing of the dividend dates, $12/f$ months (f, Step 1)
`market_price`	`Market_Price`	Yes	Starting price and base of all cash flows (MP, Steps 3 and 5)
`growth_rate`	`Growth_Rate`	Yes	Growth of the price and of the cash flows (g, Steps 3 and 5)
`dividend_yield`	`Dividend_Yield`	Yes	Dividend paid on each dividend date, as a fraction of the market value (dy, Step 3)
`units`	`Units`	Yes	Starting holding; scales the cash flows and market value, and changes with trading (u, Steps 4 and 6)
`spread_country`, `spread_sector`, `spread_stress`	`Spread_Country`, `Spread_Sector`, `Spread_Stress`	No	Used by `price_share` only (Step 7)
`nace`	`NACE`	No	Sector code; stored but not used
`issuer`	–	No	Not read from the CSV; always `None`

1.5 4. Steps 1 to 3 in numbers: cash flow dates and amounts

This section builds the dividend and terminal schedules of Step 1, with the year fractions of Step 2 and the amounts of Step 3, for every share. `main.py` does this once, before the main loop starts.

1.5.1 Dividend and terminal flows

The dividends and the terminal value are simulated separately.

`create_dividend_flows` returns the dividend dates and amounts of every share, as a dictionary keyed by asset id:

[14]:
```
div_dict = eq_ptf.create_dividend_flows(modelling_date=settings.modelling_date,␣
↪end_date=settings.end_date)
```

`create_terminal_flows` returns the terminal values in the same shape:

[15]:
```
ter_dict = eq_ptf.create_terminal_flows(modelling_date=settings.modelling_date,
                                        terminal_date=settings.end_date,
                                        terminal_rate=curves.ufr)
```

A compact view of the same two dictionaries, one row per share. The dividend grows from one payment to the next because the market value does, and the terminal value is paid on the end of the modelling window, E:

---

[16]:
```
dividend_schedule = pd.DataFrame({
    "Dividends": {aid: len(flows) for aid, flows in div_dict.items()},
    "First dividend date": {aid: min(flows) for aid, flows in div_dict.items()},
    "First dividend": {aid: flows[min(flows)] for aid, flows in div_dict.
 ↪items()},
    "Last dividend date": {aid: max(flows) for aid, flows in div_dict.items()},
    "Last dividend": {aid: flows[max(flows)] for aid, flows in div_dict.
 ↪items()},
    "Terminal date": {aid: min(flows) for aid, flows in ter_dict.items()},
    "Terminal value": {aid: flows[min(flows)] for aid, flows in ter_dict.
 ↪items()},
})

display(dividend_schedule)
```

	Dividends	First dividend date	First dividend	Last dividend date	\
1125	50	2023-12-03	3.712281	2072-12-03	
2123	50	2023-12-03	0.613205	2072-12-03	
3232	50	2023-12-03	2.315166	2072-12-03	
1	50	2023-12-03	2.323760	2072-12-03	
2	50	2023-12-03	3.946368	2072-12-03	
3	50	2023-12-03	0.050365	2072-12-03	
4	50	2023-12-03	0.094158	2072-12-03	
5	50	2023-12-03	2.282822	2072-12-03	
6	50	2023-12-03	3.930951	2072-12-03	
7	50	2023-12-03	2.836797	2072-12-03	
8	50	2023-12-03	4.654691	2072-12-03	
9	50	2023-12-03	3.930951	2072-12-03	
10	50	2023-12-03	2.836797	2072-12-03	
11	50	2023-12-03	4.654691	2072-12-03	
12	50	2023-12-03	3.930951	2072-12-03	
13	50	2023-12-03	2.836797	2072-12-03	
14	50	2023-12-03	4.654691	2072-12-03	
15	50	2023-12-03	3.930951	2072-12-03	
16	50	2023-12-03	2.836797	2072-12-03	
17	50	2023-12-03	4.654691	2072-12-03	
18	50	2023-12-03	3.930951	2072-12-03	

19	50	2023-12-03	2.836797	2072-12-03
20	50	2023-12-03	4.654691	2072-12-03
21	50	2023-12-03	3.930951	2072-12-03
22	50	2023-12-03	2.836797	2072-12-03
23	50	2023-12-03	4.654691	2072-12-03
24	50	2023-12-03	3.930951	2072-12-03
25	50	2023-12-03	2.836797	2072-12-03
26	50	2023-12-03	4.654691	2072-12-03
27	50	2023-12-03	3.930951	2072-12-03
28	50	2023-12-03	2.836797	2072-12-03
29	50	2023-12-03	4.654691	2072-12-03
30	50	2023-12-03	3.957567	2072-12-03

	Last dividend	Terminal date	Terminal value
1125	6.045011	2073-04-29	202.308916
2123	1.618198	2073-04-29	32.622931
3232	15.821613	2073-04-29	401.833450
1	3.783967	2073-04-29	126.638375
2	10.414147	2073-04-29	209.949558
3	0.344191	2073-04-29	8.741690
4	0.153324	2073-04-29	5.131321
5	6.024184	2073-04-29	121.447744
6	26.863723	2073-04-29	682.278232
7	4.619389	2073-04-29	154.597497
8	12.283353	2073-04-29	247.632811
9	26.863723	2073-04-29	682.278232
10	4.619389	2073-04-29	154.597497
11	12.283353	2073-04-29	247.632811
12	26.863723	2073-04-29	682.278232
13	4.619389	2073-04-29	154.597497
14	12.283353	2073-04-29	247.632811
15	26.863723	2073-04-29	682.278232
16	4.619389	2073-04-29	154.597497
17	12.283353	2073-04-29	247.632811
18	26.863723	2073-04-29	682.278232
19	4.619389	2073-04-29	154.597497
20	12.283353	2073-04-29	247.632811
21	26.863723	2073-04-29	682.278232
22	4.619389	2073-04-29	154.597497
23	12.283353	2073-04-29	247.632811
24	26.863723	2073-04-29	682.278232
25	4.619389	2073-04-29	154.597497
26	12.283353	2073-04-29	247.632811
27	26.863723	2073-04-29	682.278232
28	4.619389	2073-04-29	154.597497
29	12.283353	2073-04-29	247.632811
30	27.045613	2073-04-29	686.897824

The first dividends of share 1125 rebuilt by hand from the formulas of Steps 2 and 3, next to the values in the schedule:

[17]:
```
example_share = eq_ptf.equity_share[1125]

by_hand = []
for dividend_date, scheduled_dividend in list(div_dict[1125].items())[:3]:
    t = (dividend_date - settings.modelling_date).days / 365.25
    market_value = example_share.market_price * (1 + example_share.growth_rate)
 ↪** t
    by_hand.append({"Date": dividend_date,
                    "Year fraction": t,
                    "Market value": market_value,
                    "Dividend by hand": market_value * example_share.
 ↪dividend_yield,
                    "Dividend in schedule": scheduled_dividend})

display(pd.DataFrame(by_hand))
```

```
         Date  Year fraction  Market value  Dividend by hand  \
0  2023-12-03       0.596851    123.742714          3.712281
1  2024-12-03       1.598905    124.982695          3.749481
2  2025-12-03       2.598220    126.231662          3.786950

   Dividend in schedule
0              3.712281
1              3.749481
2              3.786950
```

1.5.2 Cash flow matrices

All cash flows can be represented in a matrix with all possible cash flow dates as columns and all shares as rows. The non-zero entries are the cash flow per share at that date. The columns are the unique payment dates across the portfolio, from `unique_dates_profile`, computed separately for dividends and terminal values:

[18]:
```
unique_div_dates = eq_ptf.unique_dates_profile(div_dict)
unique_ter_dates = eq_ptf.unique_dates_profile(ter_dict)
```

`create_cashflow_dataframe` then converts the dictionaries of dates and amounts into a single DataFrame each:

[19]:
```
# Dataframe with equity dividend cash flows
```

```
div_df = create_cashflow_dataframe(cf_dict=div_dict,␣
 ↪unique_dates=unique_div_dates)
# Dataframe with equity terminal cash flows
ter_df = create_cashflow_dataframe(cf_dict=ter_dict,␣
 ↪unique_dates=unique_ter_dates)
```

Cash flow dataframe with dividend amounts and dates:

---

[20]:
```
display(div_df.head())
```

```
      2023-12-03  2024-12-03  2025-12-03  2026-12-03  2027-12-03  2028-12-03  \
1125    3.712281    3.749481    3.786950    3.824793    3.863015    3.901725
2123    0.613205    0.625494    0.637996    0.650747    0.663753    0.677055
3232    2.315166    2.407967    2.504218    2.604317    2.708417    2.816981
1       2.323760    2.347045    2.370499    2.394188    2.418114    2.442345
2       3.946368    4.025460    4.105913    4.187975    4.271676    4.357287

      2029-12-03  2030-12-03  2031-12-03  2032-12-03  …  2063-12-03  \
1125    3.940715    3.980095    4.019869    4.060150  …    5.527081
2123    0.690587    0.704389    0.718467    0.732866  …    1.353981
3232    2.929581    3.046683    3.168465    3.295469  …   11.115161
1       2.466751    2.491402    2.516299    2.541514  …    3.459761
2       4.444372    4.533198    4.623800    4.716467  …    8.713738

      2064-12-03  2065-12-03  2066-12-03  2067-12-03  2068-12-03  2069-12-03  \
1125    5.582466    5.638252    5.694596    5.751503    5.809137    5.867188
2123    1.381117    1.408720    1.436875    1.465592    1.494965    1.524844
3232   11.560698   12.022803   12.503380   13.003166   13.524382   14.064979
1       3.494430    3.529351    3.564620    3.600242    3.636318    3.672656
2       8.888374    9.066019    9.247214    9.432030    9.621062    9.813350

      2070-12-03  2071-12-03  2072-12-03
1125    5.925819    5.985037    6.045011
2123    1.555319    1.586404    1.618198
3232   14.627186   15.211865   15.821613
1       3.709358    3.746426    3.783967
2      10.009482   10.209533   10.414147

[5 rows x 50 columns]
```

Cash flow dataframe with terminal amounts and dates:

---

[21]:
```
display(ter_df.head())
```

```
      2073-04-29
1125  202.308916
2123   32.622931
3232  401.833450
1     126.638375
2     209.949558
```

1.6 5. Steps 4 to 6 in numbers: the main loop

This section runs the equity leg of the main loop: Step 4, Step 5 and Step 6 over all projection dates.

1.6.1 Running the equity leg

The cell below makes the same calls in the same order as the main loop in `main.py`, with everything that is not equity removed: the bond DataFrames are empty, there are no liabilities and the bank account starts at zero. All the cash in the bank account therefore comes from the shares, and `trade` reinvests it in them. With negligible bond holdings, no liabilities and no starting cash in the input files, `main.py` writes these same figures to `Results.csv`.

`init_equity_portfolio_to_dataframe` sets up the three DataFrames the loop works on, each with one column for the modelling date: the price of each share, its growth rate and the units held. The price starts at `Market_Price`.

[22]:
```
eq_price_df, eq_growth_df, eq_units_df = eq_ptf.
 ↪init_equity_portfolio_to_dataframe(modelling_date=settings.modelling_date)

# The loop consumes the cash flow matrices, so keep them for the pricing in␣
 ↪section 6
div_df_t0, ter_df_t0 = div_df.copy(), ter_df.copy()
```

[23]:
```
dates_of_interest = set_dates_of_interest(settings.modelling_date, settings.
 ↪end_date)

# No bonds and no starting cash in this notebook
bd_price_df = pd.DataFrame(index=pd.Index([], dtype=int), columns=[settings.
 ↪modelling_date], dtype=float)
bd_units_df = bd_price_df.copy()
bank_account = pd.DataFrame({settings.modelling_date: [0.0]})

summary = {}
previous_date = settings.modelling_date
prev_mkt_value = portfolio_market_value(eq_price_df, eq_units_df, bd_price_df,␣
 ↪bd_units_df, previous_date)

for current_date in dates_of_interest.values:
    # Carry forward from the previous period
```

```
    eq_units_df[current_date] = eq_units_df[previous_date]
    bd_units_df[current_date] = bd_units_df[previous_date]
    bd_price_df[current_date] = bd_price_df[previous_date]
    bank_account[current_date] = bank_account[previous_date]
    time_frac = (current_date - previous_date).days / 365.25

    # Step 4: credit the cash flows that have come due
    dividend_cash, div_df, unique_div_dates =␣
↪process_expired_cf(unique_dates=unique_div_dates,␣
↪expiration_date=current_date, cash_flows=div_df, units=eq_units_df)
    bank_account[current_date] += dividend_cash
    terminated_eq_ids =␣
↪find_terminated_positions(unique_dates=unique_ter_dates,␣
↪expiration_date=current_date, cash_flows=ter_df)
    terminal_cash, ter_df, unique_ter_dates =␣
↪process_expired_cf(unique_dates=unique_ter_dates,␣
↪expiration_date=current_date, cash_flows=ter_df, units=eq_units_df)
    bank_account[current_date] += terminal_cash

    # Step 5: grow the price, then measure the period return
    eq_price_df[current_date] = eq_price_df[previous_date] * (1 +␣
↪eq_growth_df[settings.modelling_date]) ** time_frac
    after_growth = portfolio_market_value(eq_price_df, eq_units_df,␣
↪bd_price_df, bd_units_df, current_date)
    portfolio_return =␣
↪portfolio_total_return(start_market_value=prev_mkt_value,␣
↪end_market_value=after_growth, asset_income=dividend_cash)

    # Step 6: close the shares that were paid out, then trade the bank account␣
↪away
    eq_units_df = liquidate_positions(units=eq_units_df,␣
↪asset_ids=terminated_eq_ids, current_date=current_date)
    eq_units_df, bd_units_df, bank_account = trade(current_date=current_date,␣
↪bank_account=bank_account, eq_units=eq_units_df, eq_price=eq_price_df,␣
↪bd_units=bd_units_df, bd_price=bd_price_df)

    end_mkt_value = portfolio_market_value(eq_price_df, eq_units_df,␣
↪bd_price_df, bd_units_df, current_date)
    summary[current_date] = {"Start market value": prev_mkt_value,
                             "Dividend cash flow": float(dividend_cash),
                             "Terminal cash flow": float(terminal_cash),
                             "After growth market value": after_growth,
                             "Portfolio return": portfolio_return,
                             "End market value": end_mkt_value,
                             "End cash": float(bank_account.loc[0,␣
↪current_date])}
```

```
    previous_date = current_date
    prev_mkt_value = end_mkt_value

summary_df = pd.DataFrame(summary).T
```

The result, using the column names of `Results.csv`; the first periods and the last ones. In each ordinary period the dividends are reinvested, so `End market value` is the after-growth value plus the dividend and `End cash` is zero, up to rounding. On the last date the terminal values are paid and the shares are closed: the market value drops to zero and everything ends up in the bank account.

---

[24]:
```
display(pd.concat([summary_df.head(4), summary_df.tail(3)]))
```

```
            Start market value  Dividend cash flow  Terminal cash flow  \
2024-04-29        2.748790e+05        11067.520563        0.000000e+00
2025-04-29        2.923471e+05        11782.190327        0.000000e+00
2026-04-29        3.109631e+05        12544.333246        0.000000e+00
2027-04-29        3.308248e+05        13358.025416        0.000000e+00
2071-04-29        6.010108e+06       249177.781792        0.000000e+00
2072-04-29        6.443036e+06       267200.224364        0.000000e+00
2073-04-29        6.908694e+06       286586.874273        7.121887e+06

            After growth market value  Portfolio return  End market value  \
2024-04-29               2.812796e+05          0.063548      2.923471e+05
2025-04-29               2.991809e+05          0.063678      3.109631e+05
2026-04-29               3.182804e+05          0.063871      3.308248e+05
2027-04-29               3.386610e+05          0.064065      3.520191e+05
2071-04-29               6.193858e+06          0.072033      6.443036e+06
2072-04-29               6.641494e+06          0.072273      6.908694e+06
2073-04-29               7.121887e+06          0.072341      0.000000e+00

                End cash
2024-04-29 -4.729372e-11
2025-04-29 -2.546585e-11
2026-04-29 -4.365575e-11
2027-04-29 -5.456968e-12
2071-04-29  9.022187e-10
2072-04-29 -1.396984e-09
2073-04-29  7.408474e+06
```

1.6.2 The price path

The price of each share on the first and last projection dates. Nothing but the growth rate moves it: dividends and trading leave it alone.

[25]:
```
display(eq_price_df.loc[[1125, 2123, 3232]].iloc[:, [0, 1, 2, 3, -2, -1]])
```

	2023-04-29	2024-04-29	2025-04-29	2026-04-29	2072-04-29	2073-04-29
1125	123.01	124.242638	125.484210	126.738189	200.307222	202.308916
2123	12.12	12.362903	12.609990	12.862015	31.983699	32.622931
3232	56.54	58.806336	61.156947	63.601518	386.388690	401.833450

Prices in the loop and cash flows from Step 3 come from the same growth formula, computed separately. The price on the last date reproduces both the terminal value and the closed form from Step 5:

[26]:
```
years_to_end = (settings.end_date - settings.modelling_date).days / 365.25

price_check = pd.DataFrame({
    "Price on last date": eq_price_df[settings.end_date],
    "Terminal value": ter_df_t0[settings.end_date],
    "Closed form": eq_price_df[settings.modelling_date] * (1 +␣
 ↪eq_growth_df[settings.modelling_date]) ** years_to_end,
})
price_check["Largest difference"] = price_check.max(axis=1) - price_check.
 ↪min(axis=1)

display(price_check.loc[[1125, 2123, 3232]])
print(f"Largest difference over the whole portfolio: {price_check['Largest␣
 ↪difference'].max():.2e}")
```

	Price on last date	Terminal value	Closed form	Largest difference
1125	202.308916	202.308916	202.308916	6.821210e-13
2123	32.622931	32.622931	32.622931	8.526513e-14
3232	401.833450	401.833450	401.833450	1.250555e-12

```
Largest difference over the whole portfolio: 2.39e-12
```

1.6.3 The units

Trading scales every position by the same factor, whatever its own dividend was. All shares in this portfolio start with 100 units, so they keep holding the same number of units as each other. The last column shows the liquidation on the last date.

[27]:
```
display(eq_units_df.loc[[1125, 2123, 3232]].iloc[:, [0, 1, 2, 3, -2, -1]])
```

	2023-04-29	2024-04-29	2025-04-29	2026-04-29	2072-04-29	2073-04-29
1125	100.0	103.934704	108.027808	112.285489	680.111862	0.0
2123	100.0	103.934704	108.027808	112.285489	680.111862	0.0
3232	100.0	103.934704	108.027808	112.285489	680.111862	0.0

1.7 6. Step 7 in numbers: discounted cash flow price

From here on the notebook leaves the path of a run. `price_share` (Step 7) and the calibration in the next section are not called by `main.py`. All pricing here happens at the modelling date, so the projection year is 0:

[28]:
```
proj_period = 0
```

1.7.1 Price against the market price

The cash flows of section 4, discounted at the risk-free curve with the input spreads, which are all zero in this portfolio. The result differs from the market price, because the growth rates and dividend yields in the input file are modelling assumptions that were not calibrated to it. A run is not affected by this difference: it starts from `Market_Price` and never computes this value.

---

[29]:
```
dcf_price = {}
for asset_id, share in eq_ptf.equity_share.items():
    dcf_price[asset_id] = share.price_share(div_df_t0.loc[asset_id].to_dict(),
                                            ter_df_t0.loc[asset_id].to_dict(),
                                            settings.modelling_date,
                                            proj_period,
                                            curves)

price_comparison = pd.DataFrame({
    "Discounted cash flow price": pd.Series(dcf_price),
    "Market price": pd.Series({aid: share.market_price for aid, share in eq_ptf.
  ↪equity_share.items()}),
})
price_comparison["Difference"] = (price_comparison["Discounted cash flow price"]
                                  - price_comparison["Market price"])

display(price_comparison)
```

	Discounted cash flow price	Market price	Difference
1125	169.974684	123.01	46.964684
2123	32.738668	12.12	20.618668
3232	247.469503	56.54	190.929503
1	106.398266	77.00	29.398266
2	210.694398	78.00	132.694398
3	5.383578	1.23	4.153578
4	4.311202	3.12	1.191202

5	121.878605	45.12	76.758605
6	420.181683	96.00	324.181683
7	129.888792	94.00	35.888792
8	248.511341	92.00	156.511341
9	420.181683	96.00	324.181683
10	129.888792	94.00	35.888792
11	248.511341	92.00	156.511341
12	420.181683	96.00	324.181683
13	129.888792	94.00	35.888792
14	248.511341	92.00	156.511341
15	420.181683	96.00	324.181683
16	129.888792	94.00	35.888792
17	248.511341	92.00	156.511341
18	420.181683	96.00	324.181683
19	129.888792	94.00	35.888792
20	248.511341	92.00	156.511341
21	420.181683	96.00	324.181683
22	129.888792	94.00	35.888792
23	248.511341	92.00	156.511341
24	420.181683	96.00	324.181683
25	129.888792	94.00	35.888792
26	248.511341	92.00	156.511341
27	420.181683	96.00	324.181683
28	129.888792	94.00	35.888792
29	248.511341	92.00	156.511341
30	423.026663	96.65	326.376663

1.7.2 Additivity of the spreads

The three spreads work in the same way, and only their sum matters. Share 1125 is priced here with the same 2% combined spread split in three different ways. Each case uses a copy of the share, since `EquityShare` instances are not changed after they are loaded.

---

[30]:
```
spread_cases = {"No spread": (0.00, 0.00, 0.00),
                "Country 1% + sector 1%": (0.01, 0.01, 0.00),
                "Country 1% + stress 1%": (0.01, 0.00, 0.01),
                "Stress 2%": (0.00, 0.00, 0.02)}

spread_prices = {}
for name, (country, sector, stress) in spread_cases.items():
    spread_share = replace(example_share, spread_country=country,␣
 ↪spread_sector=sector, spread_stress=stress)
    spread_prices[name] = spread_share.price_share(div_df_t0.loc[1125].
 ↪to_dict(),
```

```
                                                    ter_df_t0.loc[1125].
 ↪to_dict(),
                                                    settings.modelling_date,
                                                    proj_period,
                                                    curves)

display(pd.Series(spread_prices, name="Discounted cash flow price").to_frame())
```

	Discounted cash flow price
No spread	169.974684
Country 1% + sector 1%	102.133451
Country 1% + stress 1%	102.133451
Stress 2%	102.133451

1.8 7. Step 8 in numbers: calibrating the growth rate

Step 8 on a single hypothetical share, with a 2% sector spread. The bisection searches the bracket $[-1, 1]$ for the growth rate at which the discounted cash flows equal the market price of 94:

[31]:
```
test_share = EquityShare(asset_id=1,
                         nace="A1.4.5",
                         issuer=None,
                         issue_date=datetime.date(2021, 12, 3),
                         dividend_yield=0.03,
                         frequency=1,
                         units=1,
                         market_price=94.0,
                         growth_rate=0.01,
                         spread_country=0,
                         spread_sector=0.02,
                         spread_stress=0)
```

[32]:
```
opt_growth = test_share.bisection_growth(x_start=-1,
                                         x_end=1,
                                         modelling_date=settings.modelling_date,
                                         end_date=settings.end_date,
                                         proj_period=proj_period,
                                         curves=curves,
                                         precision=1e-8,
                                         max_iter=100000)
```

[33]:
```
test_dividends = test_share.create_single_cash_flows(settings.modelling_date,␣
 ↪settings.end_date, opt_growth)
test_terminal = test_share.create_single_terminal(settings.modelling_date,␣
 ↪settings.end_date, curves.ufr, opt_growth)
```

Repricing the share at the calibrated growth rate reproduces its market price. The residual is the

price cost of stopping the bisection once the bracket is narrower than `precision`, as described in Step 8:

---

[34]:
```
repriced_value = test_share.price_share(test_dividends, test_terminal, settings.
↪modelling_date, proj_period, curves)

display(pd.Series({"Calibrated growth rate": opt_growth,
                   "Repriced value": repriced_value,
                   "Market price": test_share.market_price,
                   "Residual": repriced_value - test_share.market_price},␣
↪name="Value").to_frame())
```

```
                          Value
Calibrated growth rate   0.017645
Repriced value          93.999982
Market price            94.000000
Residual                -0.000018
```

1.9 8. Appendix: input data

The input files this notebook reads, all in the `Input` folder. Their locations come from the `[INPUT]` section of `ALM.ini`, except the EIOPA files, whose paths are set in `Parameters.csv`. Dates are written as `dd/mm/yyyy`.

1.9.1 Parameters.csv

The type of run and its parameters, one `Parameter`, `Value` pair per row. Loaded into `Settings` by `get_settings`.

Parameter	Example	Meaning
`EIOPA_param_file`	`Input/Param_no_VA.csv`	Relative path of the EIOPA parameter file
`EIOPA_curves_file`	`Input/Curves_no_VA.csv`	Relative path of the EIOPA spot curve file
`country`	`Slovenia`	Country whose curve is used; must match a column name in both EIOPA files
`run_type`	`Risk Neutral`	Type of run. Read into `Settings` but not used yet
`n_proj_years`	`50`	Length of the run in years. The end of the modelling window E is the modelling date plus this many years

Parameter	Example	Meaning
`Precision`	`1E-10`	Precision of the Smith-Wilson calibration in `Curves`. The growth calibration does not use it: `bisection_growth` takes its own `precision` argument
`Tau`	`0.0001`	Accepted gap between the extrapolated forward rate and the UFR at the convergence point (1 bp)
`compounding`	`-1`	Compounding convention. Read into `Settings` but not used yet; discounting is always annual, $(1+y)^{-t}$
`Modelling_Date`	`29/04/2023`	Start date of the run, MD
`liability_mode`	`cashflow`	`cashflow` or `unit_linked`; selects the liability model of the full run. Not used in this notebook
`random_seed`	`42`	Seed for the random draws in `unit_linked` mode. Not used in this notebook

1.9.2 EIOPA RFR files

Two files derived from EIOPA's monthly risk-free rate (RFR) publication. `import_SWEiopa` reads the columns of the chosen `country` from each. The `no_VA` files hold the curves without the volatility adjustment.

- **`Curves_no_VA.csv`**: spot rates, one row per maturity (1 to 150 years) and one column per country, as decimals (`0.03472` is 3.472%).
- **`Param_no_VA.csv`**: the parameters EIOPA used to build the curves, in two columns per country (`<Country>_Maturities`, `<Country>_Values`). The first six rows are scalar parameters: `Coupon_freq`, `LLP` (last liquid point), `Convergence` (convergence period), `UFR` (ultimate forward rate, in percent, e.g. `3.45`), `alpha` (speed of convergence to the UFR) and `CRA` (credit risk adjustment, in bp). The rows after them hold the Smith-Wilson calibration vector by maturity.

Together they give the risk free term structure at the modelling date, from which `Curves` projects its evolution over the run.

1.9.3 Equity_Portfolio.csv

The equity portfolio, one row per position. Loaded by `get_equity_share`, one `EquityShare` per row. Section 3 shows which of these fields affect a run.

Column	Example	Meaning
`Asset_ID`	`1125`	Unique integer id of the position
`Asset_Type`	`Equity_Share`	Asset type label; not read by `get_equity_share`
`NACE`	`A1.4.5`	NACE code of the issuer (*nomenclature statistique des activités économiques dans la Communauté européenne*)
`Issue_Date`	`3/12/2021`	Issue date I; anchors the dividend grid
`Dividend_Yield`	`0.03`	Yield dy paid on **each** dividend date, as a fraction of the market value. Not annualised: see the note on f under Notation
`Frequency`	`1`	Dividend payments per year f: 1, 2, 3, 4 or 12
`Units`	`100`	Number of shares held at the modelling date
`Market_Price`	`123.01`	Price of one share at the modelling date, MP
`Terminal`	`1`	Not read by `get_equity_share`. The column name starts with a space in the file
`Default_Probability`	`0.03`	Not read by `get_equity_share`
`Growth_Rate`	`0.01`	Annual growth rate g of the price
`Spread_Base`	`0`	Not read by `get_equity_share`
`Spread_Individual`	`0`	Not read by `get_equity_share`
`Spread_Country`	`0`	Country spread; used by `price_share` only
`Spread_Sector`	`0`	Sector spread; used by `price_share` only
`Spread_Stress`	`0`	Spread of an assumed shock scenario; used by `price_share` only